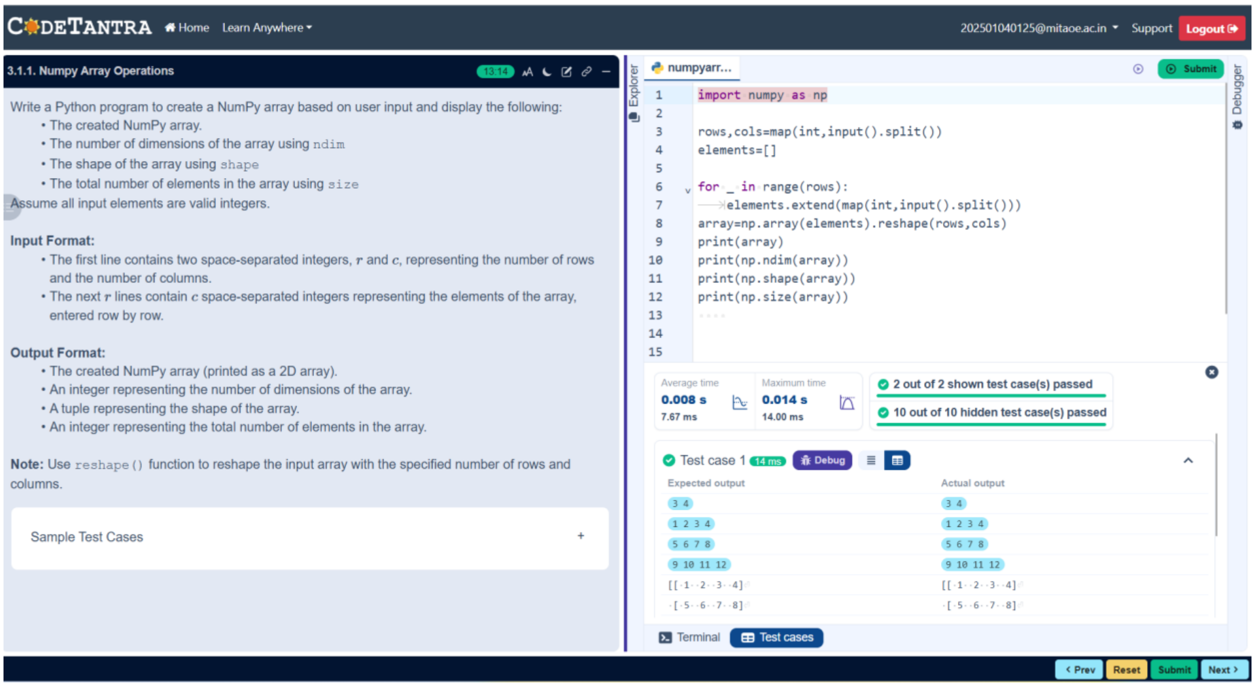
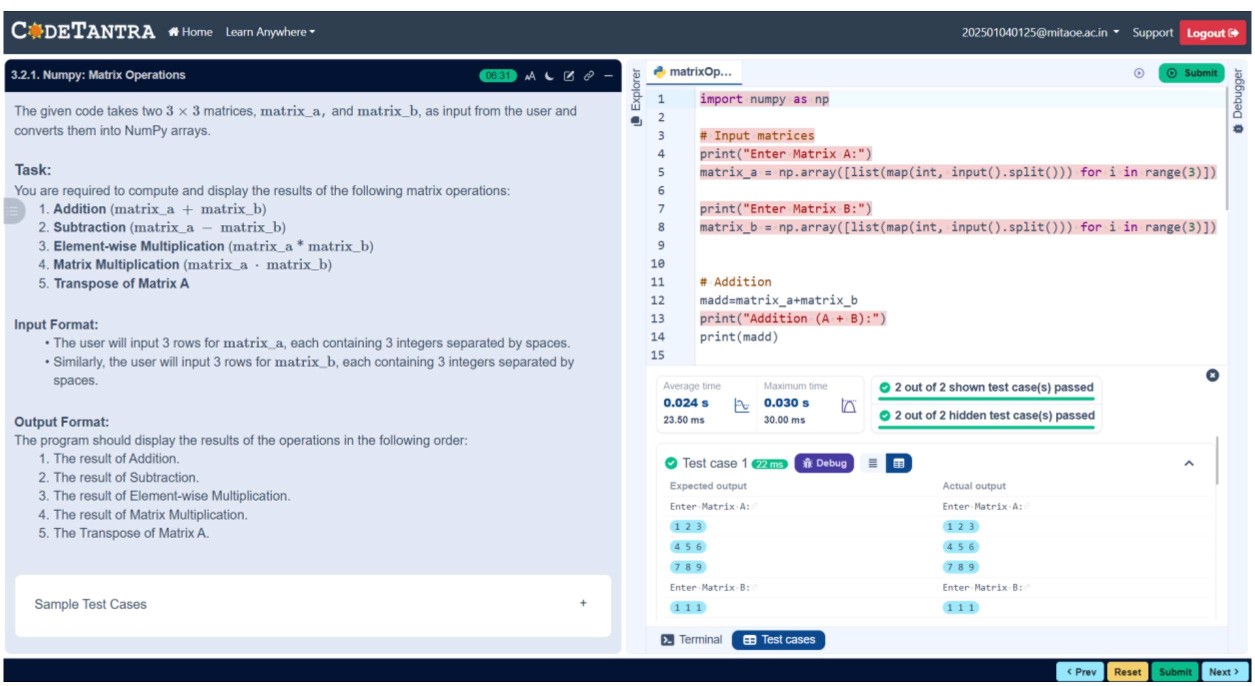
Name: Janhvi Sawarkar

PRN: 202501040125

Batch: CS2(C21)

CODETANTRA LAB PRACTICAL 3





CODETANTRA Home Learn Anywhere 202501040125@mitaoe.ac.in Support Logout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays 08:38

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking**: Stack the two matrices horizontally (side by side).
- **Vertical Stacking**: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases +

stacking.py Submit

```python
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a=np.hstack((arr1,arr2))
print("Horizontal Stack:")
print(a)

# Perform vertical stacking (vstack)
```

Average time	Maximum time
0.019 s 19.00 ms	0.029 s 29.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 29 ms Debug

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:
4 5 6	4 5 6

Terminal Test cases

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501040125@mitaoe.ac.in Support Logout

3.2.3. Numpy: Custom Sequence Generation 06:36

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases +

customS... Submit

```python
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
a=np.arange(start,stop,step)
print(a)


# Print the generated sequence

```

Average time	Maximum time
0.009 s 8.50 ms	0.011 s 11.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

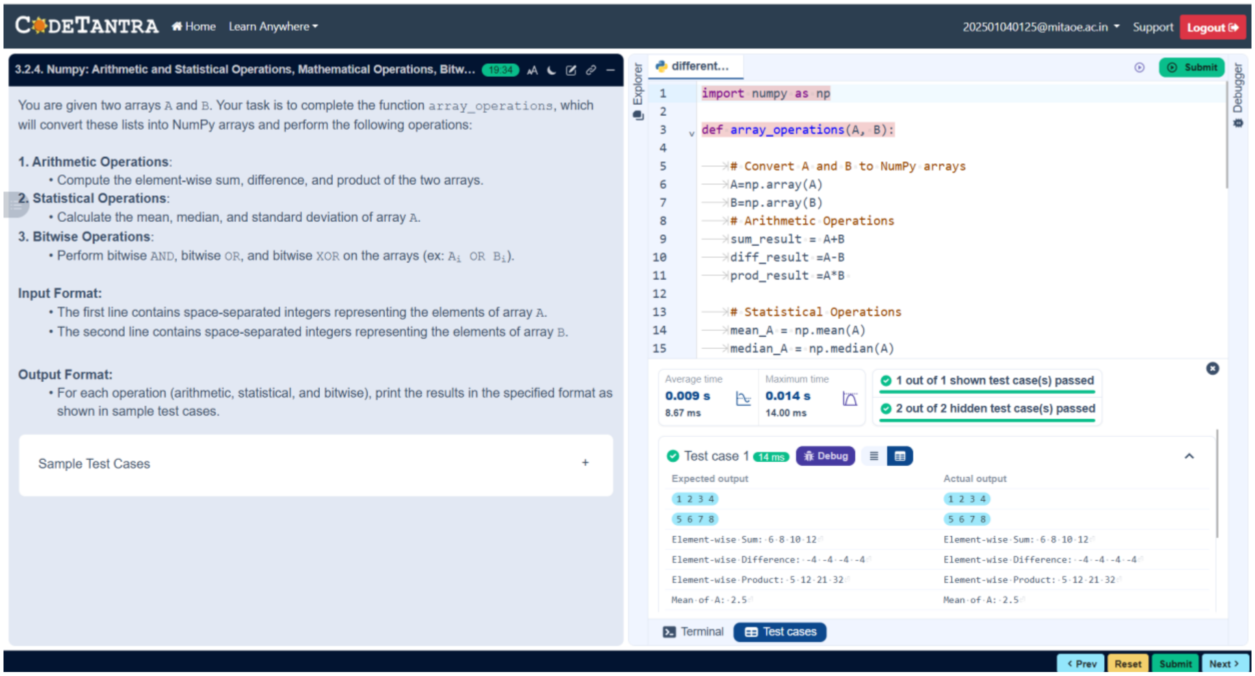
Test case 1 11 ms Debug

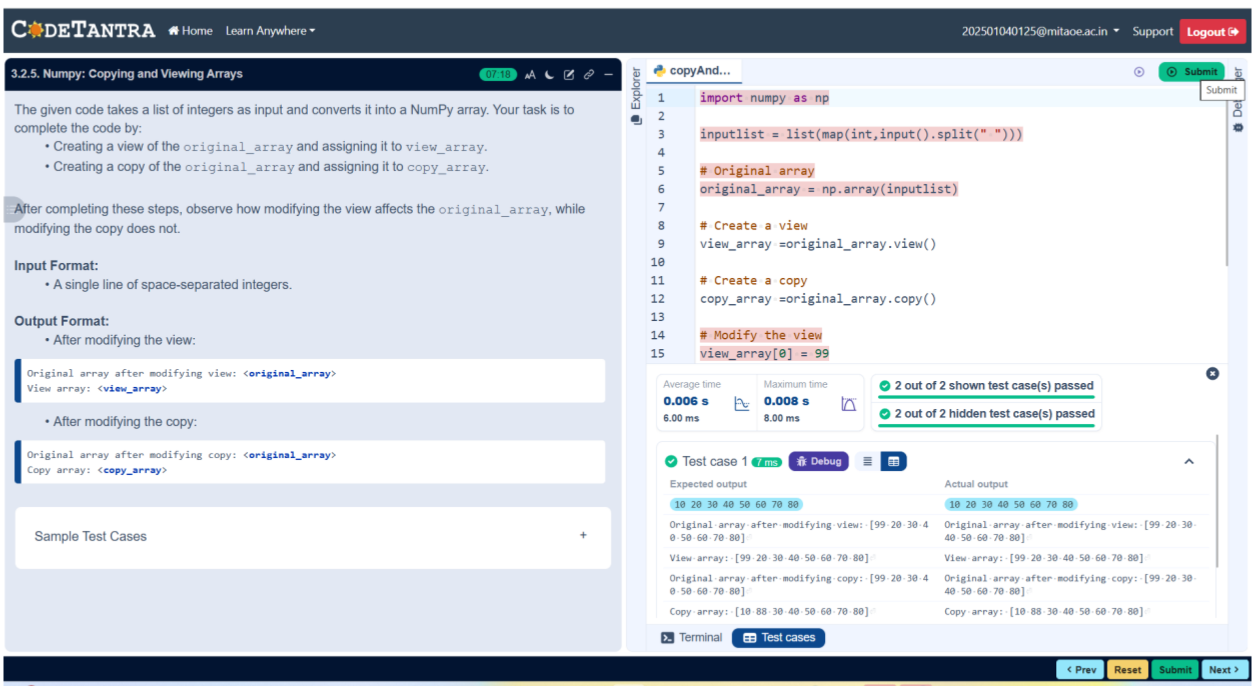
Expected output	Actual output
3	3
10	10
2	2
[3 5 7 9]	[3 5 7 9]

Test case 2 9 ms

Terminal Test cases

< Prev Reset Submit Next >


CODETANTRA
Home
Learn Anywhere
202501040125@mitaoe.ac.in
Support
Logout
3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitw...
19:34
You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:
1. Arithmetic Operations:
• Compute the element-wise sum, difference, and product of the two arrays.
2. Statistical Operations:
• Calculate the mean, median, and standard deviation of array A.
3. Bitwise Operations:
• Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A_i OR B_i).
Input Format:
• The first line contains space-separated integers representing the elements of array A.
• The second line contains space-separated integers representing the elements of array B.
Output Format:
• For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.
Sample Test Cases
Explorer
different...
Submit
Debugger
import numpy as np
def array_operations(A, B):
# Convert A and B to NumPy arrays
A=np.array(A)
B=np.array(B)
# Arithmetic Operations
sum_result = A+B
diff_result =A-B
prod_result =A*B
# Statistical Operations
mean_A = np.mean(A)
median_A = np.median(A)
Average time
0.009 s
8.67 ms
Maximum time
0.014 s
14.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
14 ms
Debug
Expected output
Actual output
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32
Mean of A: 2.5
Terminal
Test cases
Prev
Reset
Submit
Next


CODETANTRA
Home
Learn Anywhere
202501040125@mitaoe.ac.in
Support
Logout
3.2.5. Numpy: Copying and Viewing Arrays
07:18
The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:
• Creating a view of the original_array and assigning it to view_array.
• Creating a copy of the original_array and assigning it to copy_array.
After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.
Input Format:
• A single line of space-separated integers.
Output Format:
• After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
• After modifying the copy:
Original array after modifying copy: <original_array>
Copy array: <copy_array>
Sample Test Cases
Explorer
copyAnd...
Submit
import numpy as np
inputlist = list(map(int,input().split(" ")))
# Original array
original_array = np.array(inputlist)
# Create a view
view_array =original_array.view()
# Create a copy
copy_array =original_array.copy()
# Modify the view
view_array[0] = 99
Average time
0.006 s
6.00 ms
Maximum time
0.008 s
8.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
7 ms
Debug
Expected output
Actual output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
Terminal
Test cases
Prev
Reset
Submit
Next

CODETANTRA Home Learn Anywhere 202501040125@mitaoe.ac.in Support Logout

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.
Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:
1. Searching: Find the indices where `search_value` appears in `array1` and print these indices.
2. Counting: Count how many times `count_value` appears in `array1` and print the count.
3. Broadcasting: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. Sorting: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Sample Test Cases +

arrayOpe... Submit

```python
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
a=np.where(array1==search_value)[0]
print(a)

# Count occurrences in array1
```

Average time 0.024 s 23.50 ms | Maximum time 0.045 s 45.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 45 ms Debug

Expected output	Actual output
1 1 1 2 2 2	1 1 1 2 2 2
Value to search: 1	Value to search: 1
Value to count: 2	Value to count: 2
Value to add: 2	Value to add: 2
[0 1 2]	[0 1 2]
3	3

Terminal Test cases

Prev Reset Submit Next

CODETANTRA Home Learn Anywhere 202501040125@mitaoe.ac.in Support Logout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details**: Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students**: Determine the total number of students in the dataset.
- **Print all student roll numbers**: Extract and print the roll numbers of all students.
- **Print Subject 1 marks**: Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2**: Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3**: Identify the highest marks in Subject 3.
- **Print all subject marks**: Display the marks of all students for each subject.
- **Find total marks of students**: Compute the total marks for each student across all subjects.
- **Find the average marks of each student**: Compute the average marks for each student.
- **Find average marks of each subject**: Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2**: Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3**: Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3**: Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2**: Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2**: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1**: Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2**: Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject**: Count the number of students

Operatio... Submit

```python
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

# 1. Print all student details

print("All student Details:\n", a  )

# 2. print total students

print("Total Students:",a.shape[0] )

# 3. Print all student Roll numbers
print("All Student Roll Nos", a[ : ,0]   )

```

Average time 0.017 s 17.00 ms | Maximum time 0.017 s 17.00 ms

1 out of 1 shown test case(s) passed

Test case 1 17 ms Debug

Expected output	Actual output
All student Details:	All student Details:
[[301. 67. 77. 88.]	[[301. 67. 77. 88.]
[302. 78. 88. 77.]	[302. 78. 88. 77.]
[303. 45. 56. 89.]	[303. 45. 56. 89.]
[304. 88. 98. 45.]	[304. 88. 98. 45.]
[305. 78. 88. 99.]	[305. 78. 88. 99.]

Terminal Test cases

Prev Reset Submit Next